AWS Cloud & Data Engineering
AWS ASSIGNMENT



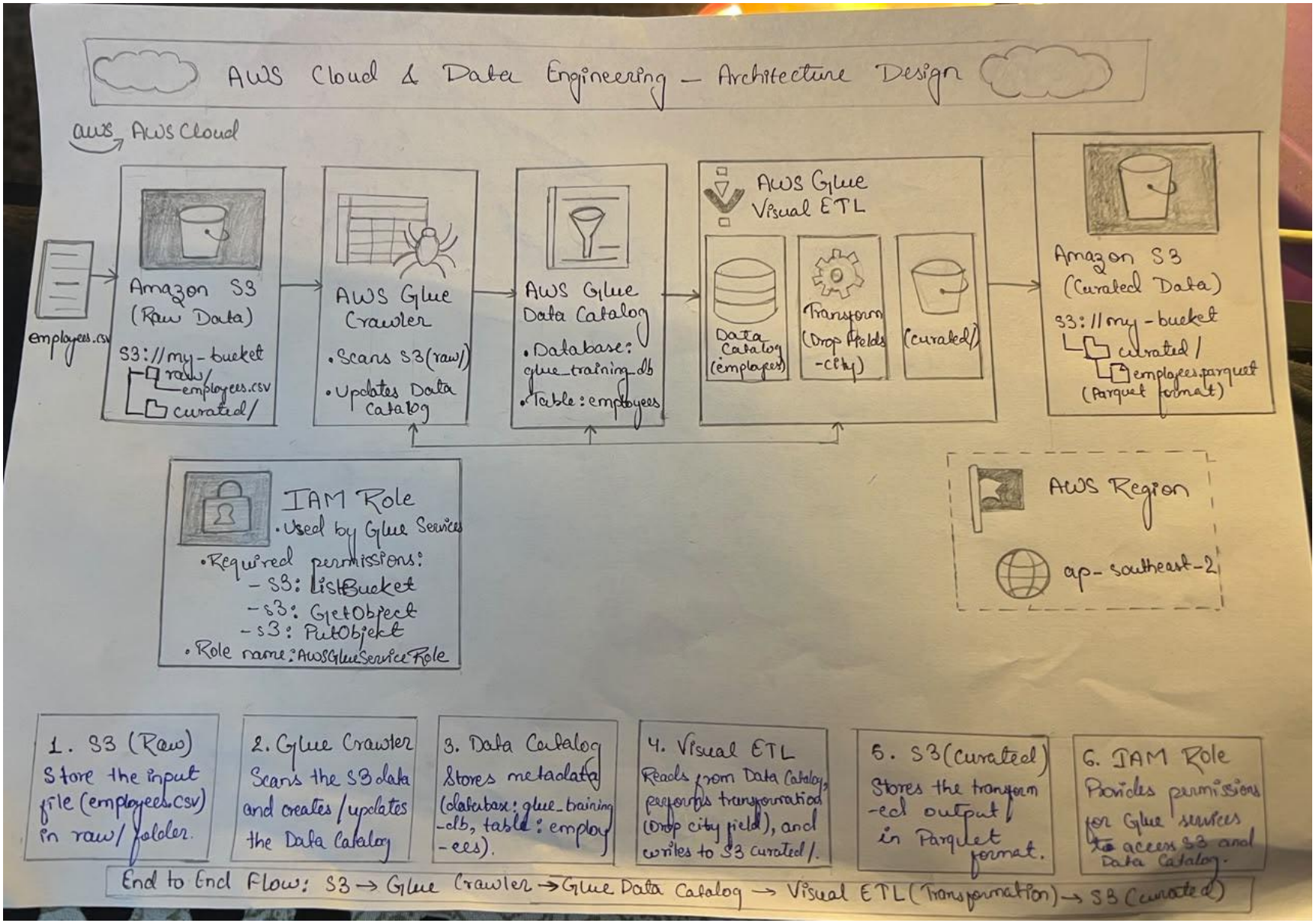
Name: Ayushi Jaiswal

U.S.N: 23BTRCF007

College Name: Jain University

Batch: CSE - CTIS

Date: 11/10/2026


AWS Cloud & Data Engineering – Architecture Design
AWS Cloud
employees.csv
Amazon S3 (Raw Data)
s3://my-bucket
raw/
employees.csv
curated/
AWS Glue Crawler
• Scans s3(raw/)
• Updates Data Catalog
AWS Glue Data Catalog
• Database: glue_training_db
• Table: employees
AWS Glue Visual ETL
Data Catalog (employees)
Transform (Drop fields -city)
(curated/)
Amazon S3 (Curated Data)
s3://my-bucket
curated/
employees.parquet
(Parquet format)
IAM Role
• Used by Glue Services
• Required permissions:
- s3: ListBucket
- s3: GetObject
- s3: PutObject
• Role name: AWSGlueServiceRole
AWS Region
ap-southeast-2
1. S3 (Raw) Store the input file (employees.csv) in raw/ folder.
2. Glue Crawler Scans the S3 data and creates/updates the Data Catalog
3. Data Catalog Stores metadata (database: glue_training_db, table: employees).
4. Visual ETL Reads from Data Catalog, performs transformation (Drop city field), and writes to S3 curated/.
5. S3 (Curated) Stores the transformed output in Parquet format.
6. IAM Role Provides permissions for Glue services to access S3 and Data Catalog.
End to End Flow: S3 → Glue Crawler → Glue Data Catalog → Visual ETL (Transformation) → S3 (curated)

SQL queries for Employee Management System

1. Create the database and employees table

```sql
CREATE DATABASE employee_management;
USE employee_management;
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(100),
    age INT,
    gender VARCHAR(10),
    department VARCHAR(50),
    job_role VARCHAR(50),
    salary DECIMAL(10,2),
    city VARCHAR(50),
    joining_date DATE
);
```

2. Insert sample employee data

```sql
INSERT INTO employees VALUES
(101, 'Rahul Sharma', 25, 'Male', 'IT', 'Developer', 45000, 'Bangalore', '2023-01-15'),
(102, 'Priya Singh', 27, 'Female', 'HR', 'HR Executive', 38000, 'Chennai', '2022-06-10'),
(103, 'Amit Kumar', 30, 'Male', 'Finance', 'Accountant', 55000, 'Delhi', '2021-03-20'),
(104, 'Sneha Reddy', 24, 'Female', 'IT', 'Tester', 40000, 'Hyderabad', '2024-02-01'),
(105, 'Karan Patel', 29, 'Male', 'Sales', 'Sales Executive', 35000, 'Mumbai', '2022-09-12'),
(106, 'Ananya Das', 28, 'Female', 'IT', 'Cloud Engineer', 65000, 'Bangalore', '2020-11-25'),
(107, 'Vikram Rao', 32, 'Male', 'Finance', 'Manager', 75000, 'Chennai', '2019-05-18'),
(108, 'Neha Verma', 26, 'Female', 'HR', 'Recruiter', 42000, 'Delhi', '2023-08-14'),
(109, 'Arjun Mehta', 31, 'Male', 'Sales', 'Sales Manager', 60000, 'Mumbai', '2020-07-30'),
(110, 'Pooja Nair', 23, 'Female', 'IT', 'Data Analyst', 48000, 'Bangalore', '2024-04-05');
```

3. SQL queries for Employee Management System

Query 1: Display all employees

```sql
SELECT * FROM employees;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

emp_id	emp_name	age	gender	department	job_role	salary	city	joining_date
101	Rahul Sharma	25	Male	IT	Developer	45000.00	Bangalore	2023-01-15
102	Priya Singh	27	Female	HR	HR Executive	38000.00	Chennai	2022-06-10
103	Amit Kumar	30	Male	Finance	Accountant	55000.00	Delhi	2021-03-20
104	Sneha Reddy	24	Female	IT	Tester	40000.00	Hyderabad	2024-02-01
105	Karan Patel	29	Male	Sales	Sales Executive	35000.00	Mumbai	2022-09-12
106	Ananya Das	28	Female	IT	Cloud Engineer	65000.00	Bangalore	2020-11-25
107	Vikram Rao	32	Male	Finance	Manager	75000.00	Chennai	2019-05-18
108	Neha Verma	26	Female	HR	Recruiter	42000.00	Delhi	2023-08-14
109	Arjun Mehta	31	Male	Sales	Sales Manager	60000.00	Mumbai	2020-07-30
110	Pooja Nair	23	Female	IT	Data Analyst	48000.00	Bangalore	2024-04-05
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Query 2: Display employee names and salaries

```
28 •  SELECT emp_name, salary
29    FROM employees;
30
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	salary
Rahul Sharma	45000.00
Priya Singh	38000.00
Amit Kumar	55000.00
Sneha Reddy	40000.00
Karan Patel	35000.00
Ananya Das	65000.00
Vikram Rao	75000.00
Neha Verma	42000.00
Arjun Mehta	60000.00
Pooja Nair	48000.00

Query 3: Find employees working in IT

```
31 •  SELECT *
32    FROM employees
33    WHERE department = 'IT';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

emp_id	emp_name	age	gender	department	job_role	salary	city	joining_date
101	Rahul Sharma	25	Male	IT	Developer	45000.00	Bangalore	2023-01-15
104	Sneha Reddy	24	Female	IT	Tester	40000.00	Hyderabad	2024-02-01
106	Ananya Das	28	Female	IT	Cloud Engineer	65000.00	Bangalore	2020-11-25
110	Pooja Nair	23	Female	IT	Data Analyst	48000.00	Bangalore	2024-04-05
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Query 4: Find employees earning more than ₹45,000

```
36 •  SELECT emp_name, salary
37    FROM employees
38    WHERE salary > 45000;
39
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	salary
Amit Kumar	55000.00
Ananya Das	65000.00
Vikram Rao	75000.00
Arjun Mehta	60000.00
Pooja Nair	48000.00

Query 5: Sort employees by salary

```
41 •    SELECT emp_name, salary
42      FROM employees
43      ORDER BY salary DESC;
44
45
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	salary
Vikram Rao	75000.00
Ananya Das	65000.00
Arjun Mehta	60000.00
Amit Kumar	55000.00
Pooja Nair	48000.00
Rahul Sharma	45000.00
Neha Verma	42000.00
Sneha Reddy	40000.00
Priya Singh	38000.00
Karan Patel	35000.00

Query 6: Count the total employees

```
46 •    SELECT COUNT(*) AS total_employees
47      FROM employees;
48
49
50
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_employees
10

Query 7: Find the average salary

```
50 •    SELECT AVG(salary) AS average_salary
51      FROM employees;
52
53
54
55
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

average_salary
50300.000000

Query 8: Find the highest salary

```
57 •    SELECT MAX(salary) AS highest_salary
58      FROM employees;
59
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

highest_salary
75000.00

Query 9: Find the lowest salary

```
61 •    SELECT MIN(salary) AS lowest_salary
62      FROM employees;
63
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

highest_salary
75000.00

Query 10: Count employees in each department

```
64 •  SELECT department, COUNT(*) AS employee_count
65    FROM employees
66    GROUP BY department;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

department	employee_count
IT	4
HR	2
Finance	2
Sales	2

Query 11: Find departments with average salary above ₹45,000

```
69 •  SELECT department, AVG(salary) AS average_salary
70    FROM employees
71    GROUP BY department
72    HAVING AVG(salary) > 45000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

department	average_salary
IT	49500.000000
Finance	65000.000000
Sales	47500.000000

Query 12: Find employees from Bangalore

```
75 •  SELECT emp_name, department, city
76    FROM employees
77    WHERE city = 'Bangalore';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	department	city
Rahul Sharma	IT	Bangalore
Ananya Das	IT	Bangalore
Pooja Nair	IT	Bangalore

Query 13: Find employees who joined after 2022

```
81 •  SELECT emp_name, joining_date
82    FROM employees
83    WHERE joining_date > '2022-12-31';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	joining_date
Rahul Sharma	2023-01-15
Sneha Reddy	2024-02-01
Neha Verma	2023-08-14
Pooja Nair	2024-04-05

Query 14: Find employees whose names start with A

```
86 •  SELECT *
87    FROM employees
88    WHERE emp_name LIKE 'A%';
89
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

emp_id	emp_name	age	gender	department	job_role	salary	city	joining_date
103	Amit Kumar	30	Male	Finance	Accountant	55000.00	Delhi	2021-03-20
106	Ananya Das	28	Female	IT	Cloud Engineer	65000.00	Bangalore	2020-11-25
109	Arjun Mehta	31	Male	Sales	Sales Manager	60000.00	Mumbai	2020-07-30
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

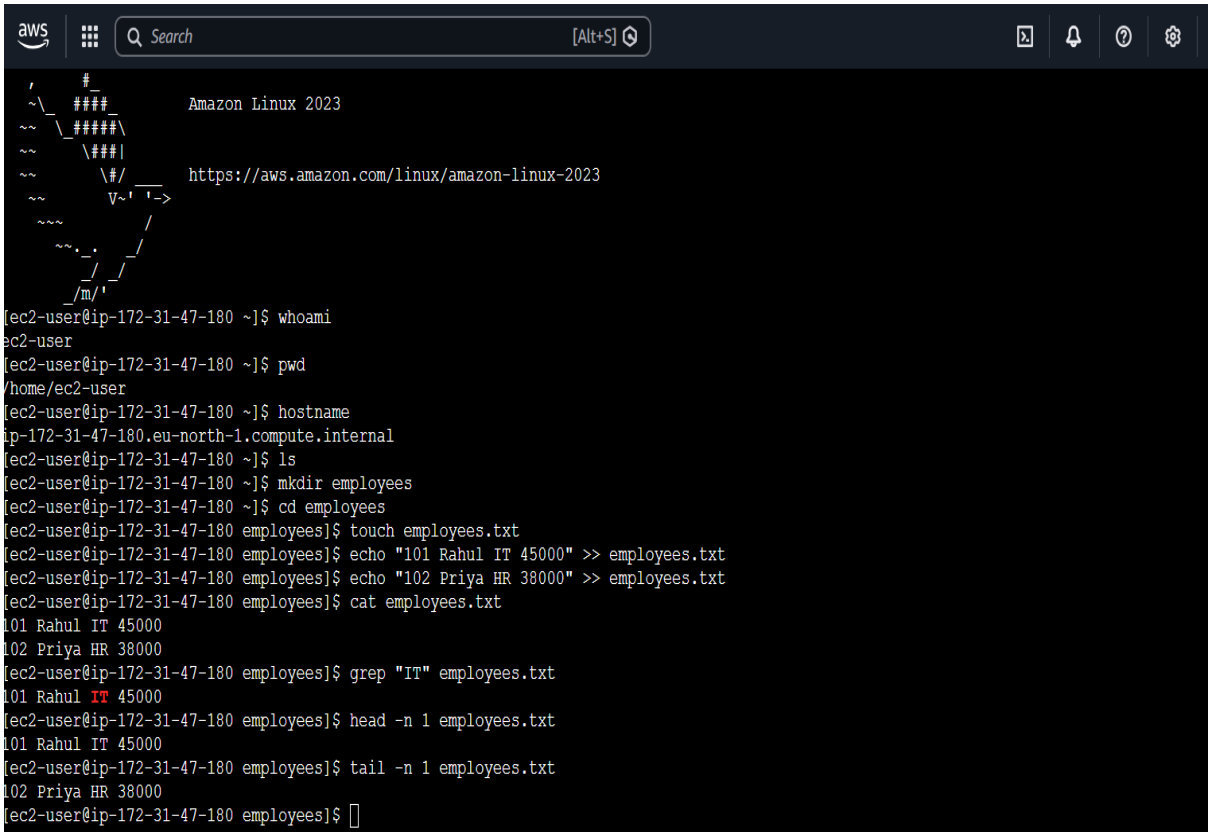
Query 15: Find employees earning between ₹40,000 and ₹60,000

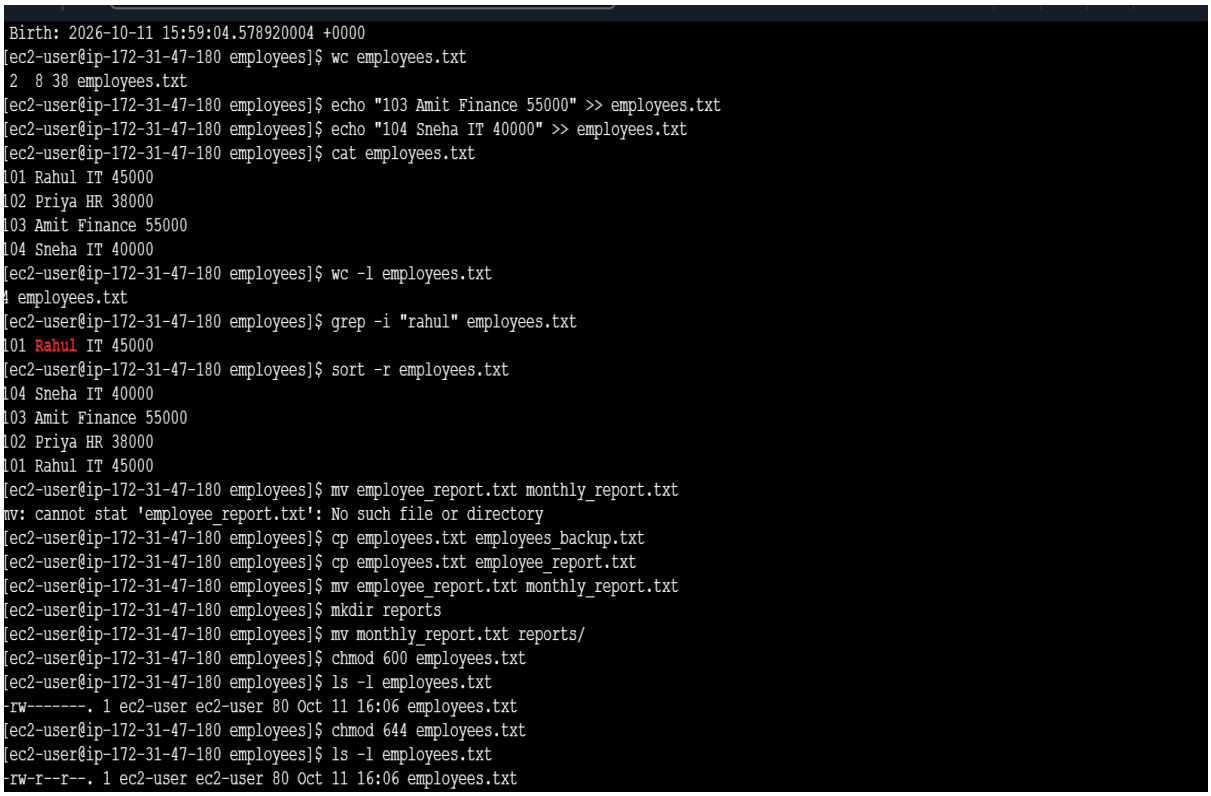
```
96 •  SELECT emp_name, salary
97    FROM employees
98    WHERE salary BETWEEN 40000 AND 60000;
99
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	salary
Rahul Sharma	45000.00
Amit Kumar	55000.00
Sneha Reddy	40000.00
Neha Verma	42000.00
Arjun Mehta	60000.00
Pooja Nair	48000.00

Linux Commands





```
[ec2-user@ip-172-31-47-180 employees]$ tail -n 1 employees.txt
102 Priya HR 38000
[ec2-user@ip-172-31-47-180 employees]$ sort employees.txt
101 Rahul IT 45000
102 Priya HR 38000
[ec2-user@ip-172-31-47-180 employees]$ grep "45000" employees.txt
101 Rahul IT 45000
[ec2-user@ip-172-31-47-180 employees]$ cp employees.txt employees_backup.txt
[ec2-user@ip-172-31-47-180 employees]$ mv employees_backup.txt employee_backup.txt
[ec2-user@ip-172-31-47-180 employees]$ ls -l employees.txt
-rw-r--r--. 1 ec2-user ec2-user 38 Oct 11 15:59 employees.txt
[ec2-user@ip-172-31-47-180 employees]$ rm employee_backup.txt
[ec2-user@ip-172-31-47-180 employees]$ ls -la
total 4
drwxr-xr-x. 2 ec2-user ec2-user 27 Oct 11 16:02 .
drwx------. 4 ec2-user ec2-user 91 Oct 11 15:58 ..
-rw-r--r--. 1 ec2-user ec2-user 38 Oct 11 15:59 employees.txt
[ec2-user@ip-172-31-47-180 employees]$ file employees.txt
employees.txt: ASCII text
[ec2-user@ip-172-31-47-180 employees]$ stat employees.txt
  File: employees.txt
  Size: 38              Blocks: 8          IO Block: 4096   regular file
Device: 10301h/66305d   Inode: 8533351     Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/ec2-user)   Gid: ( 1000/ec2-user)
Context: unconfined_u:object_r:user_home_t:s0
Access: 2026-10-11 15:59:04.578920004 +0000
Modify: 2026-10-11 15:59:55.639071638 +0000
Change: 2026-10-11 15:59:55.639071638 +0000
 Birth: 2026-10-11 15:59:04.578920004 +0000
[ec2-user@ip-172-31-47-180 employees]$ wc employees.txt
 2  8 38 employees.txt
[ec2-user@ip-172-31-47-180 employees]$
```

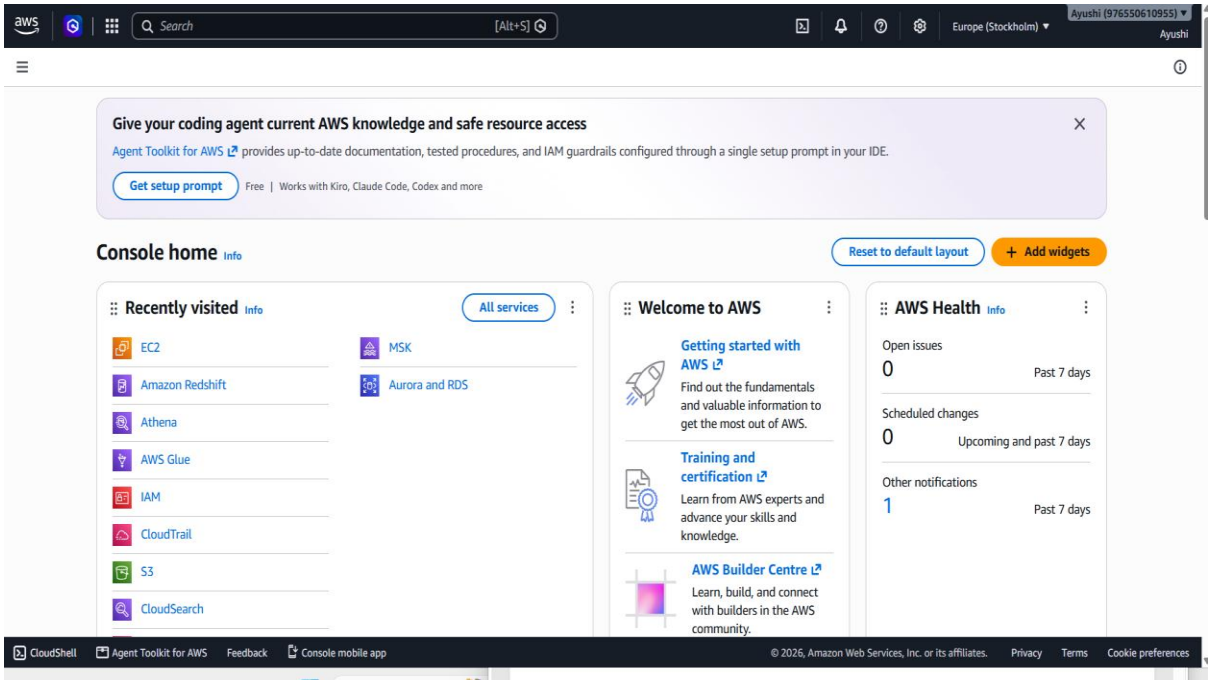
```
[ec2-user@ip-172-31-47-180 employees]$ history
    1  whoami
    2  pwd
    3  hostname
    4  ls
    5  mkdir employees
    6  cd employees
    7  touch employees.txt
    8  echo "101 Rahul IT 45000" >> employees.txt
    9  echo "102 Priya HR 38000" >> employees.txt
   10  cat employees.txt
   11  grep "IT" employees.txt
   12  head -n 1 employees.txt
   13  tail -n 1 employees.txt
   14  sort employees.txt
   15  grep "45000" employees.txt
   16  cp employees.txt employees_backup.txt
   17  mv employees_backup.txt employee_backup.txt
   18  ls -l employees.txt
   19  rm employee_backup.txt
   20  ls -la
   21  file employees.txt
   22  stat employees.txt
   23  wc employees.txt
   24  echo "103 Amit Finance 55000" >> employees.txt
   25  echo "104 Sneha IT 40000" >> employees.txt
   26  cat employees.txt
   27  wc -l employees.txt
   28  grep -i "rahul" employees.txt
   29  sort -r employees.txt
   30  mv employee_report.txt monthly_report.txt
   31  cp employees.txt employees_backup.txt
```

Linux Commands – File and Directory Management

Linux commands were used to navigate directories, create folders and files, insert employee records, and perform basic file operations. Commands such as pwd, ls, mkdir, cd, touch, and echo helped manage files and directories. The cat, grep, wc, head, tail, and sort commands were used to view, search, count, and organize employee records. Additionally, cp, mv, and rm were used to copy, rename, and delete files, demonstrating basic Linux command-line operations.

AWS Account/Region

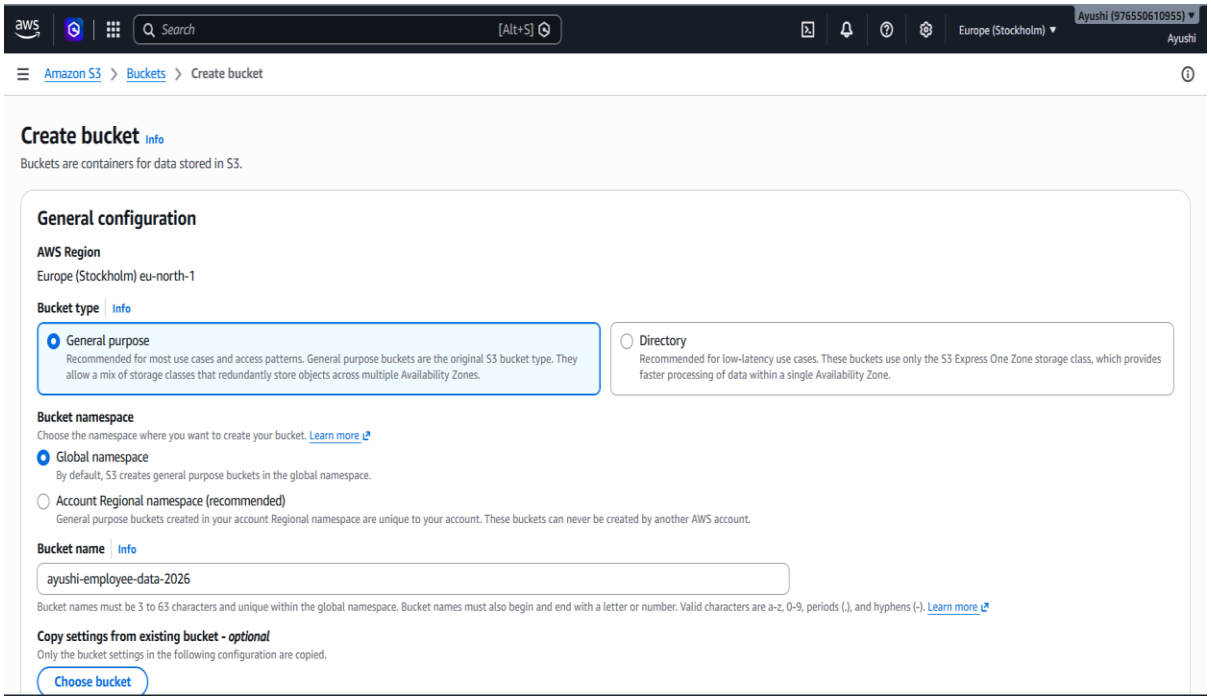
- **AWS Management Console**

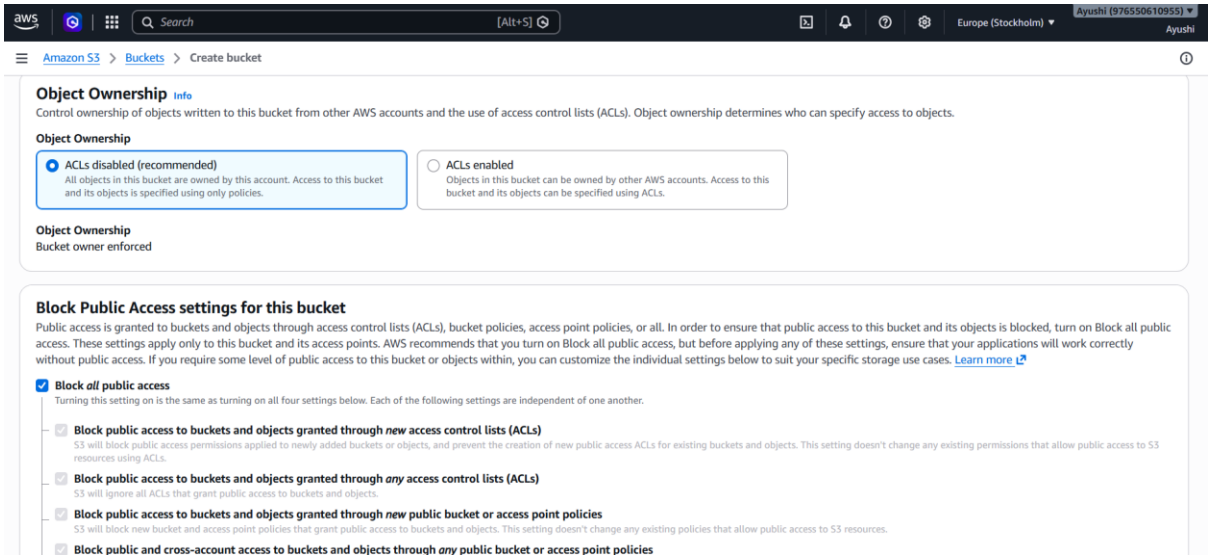


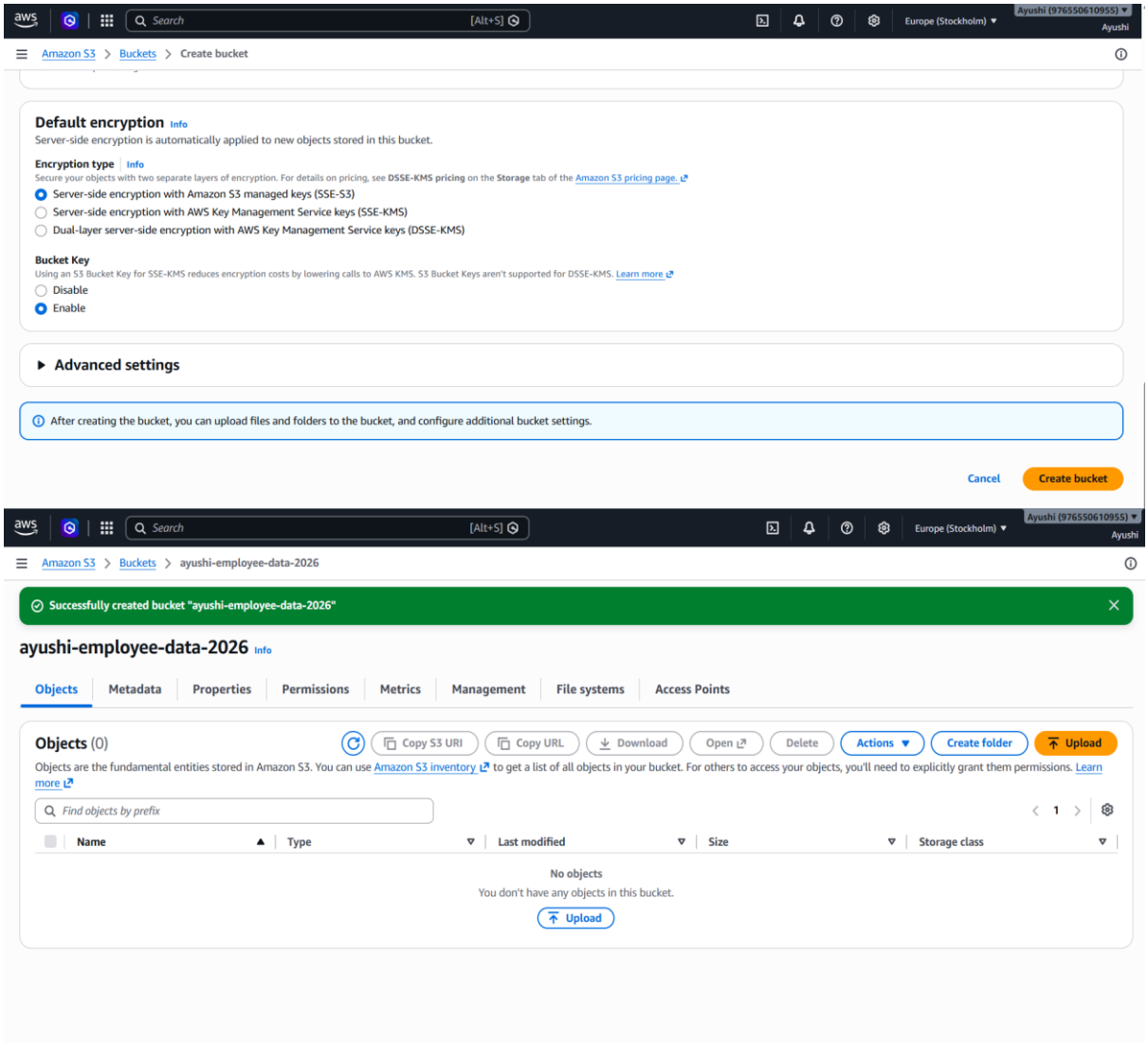
The AWS Management Console provides access to cloud services used in the assignment. The selected AWS region determines where regional resources are created and managed.

Amazon S3

- **Creating and configuring an S3 bucket**

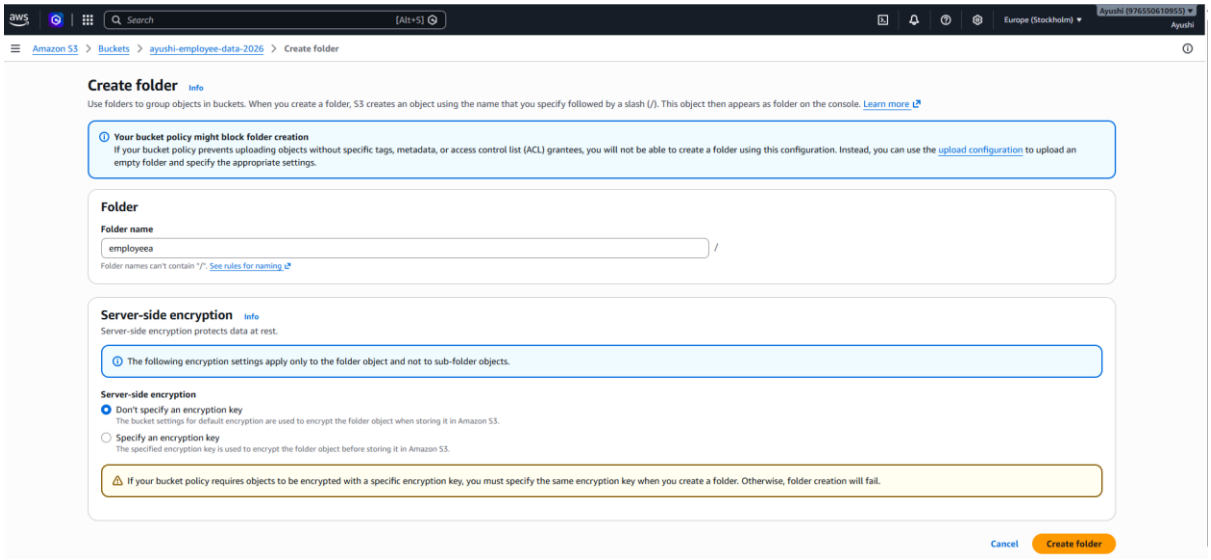

Search
[Alt+S]
Europe (Stockholm)
Ayushi (976550610955)
Ayushi
Amazon S3 > Buckets > Create bucket
Create bucket Info
Buckets are containers for data stored in S3.
General configuration
AWS Region
Europe (Stockholm) eu-north-1
Bucket type Info
General purpose
Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.
Directory
Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.
Bucket namespace
Choose the namespace where you want to create your bucket. Learn more
Global namespace
By default, S3 creates general purpose buckets in the global namespace.
Account Regional namespace (recommended)
General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by another AWS account.
Bucket name Info
ayushi-employee-data-2026
Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). Learn more
Copy settings from existing bucket - optional
Only the bucket settings in the following configuration are copied.
Choose bucket

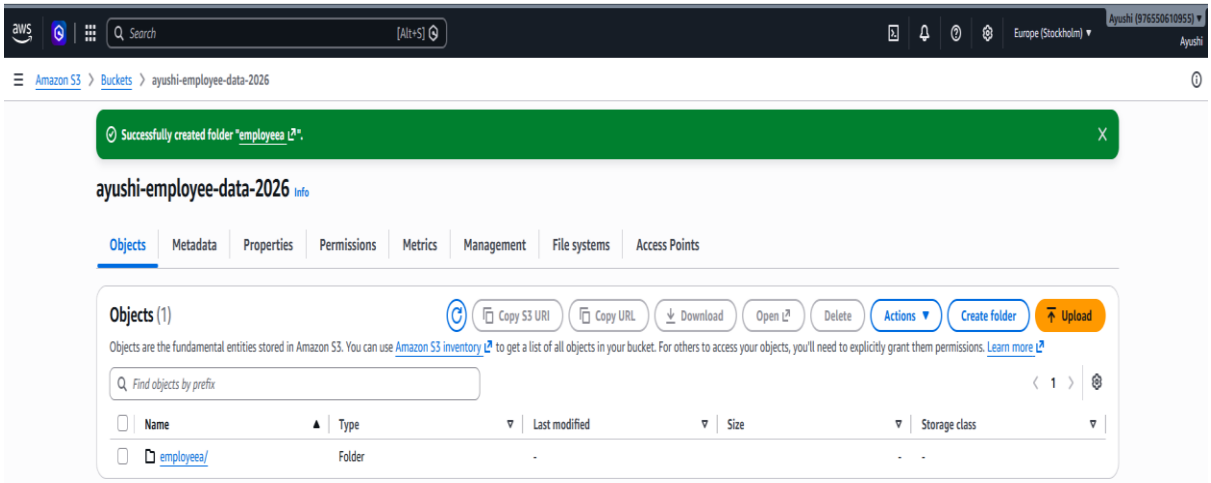

Search
[Alt+S]
Europe (Stockholm)
Ayushi (976550610955)
Ayushi
Amazon S3 > Buckets > Create bucket
Object Ownership Info
Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.
Object Ownership
ACLs disabled (recommended)
All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.
ACLs enabled
Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.
Object Ownership
Bucket owner enforced
Block Public Access settings for this bucket
Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. Learn more
Block all public access
Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.
Block public access to buckets and objects granted through new access control lists (ACLs)
S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.
Block public access to buckets and objects granted through any access control lists (ACLs)
S3 will ignore all ACLs that grant public access to buckets and objects.
Block public access to buckets and objects granted through new public bucket or access point policies
S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.
Block public and cross-account access to buckets and objects through any public bucket or access point policies

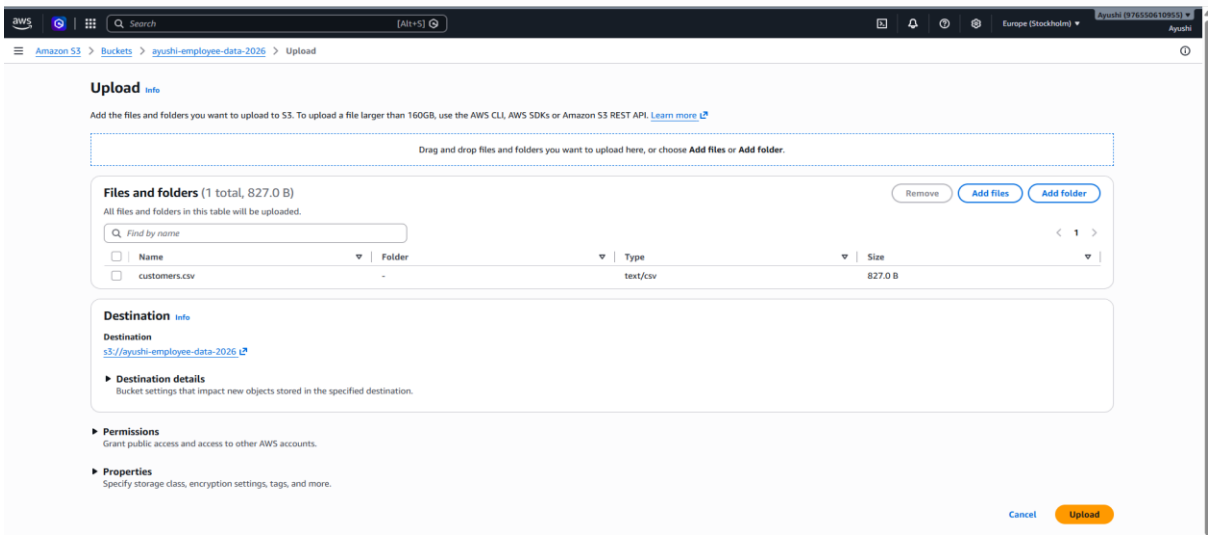


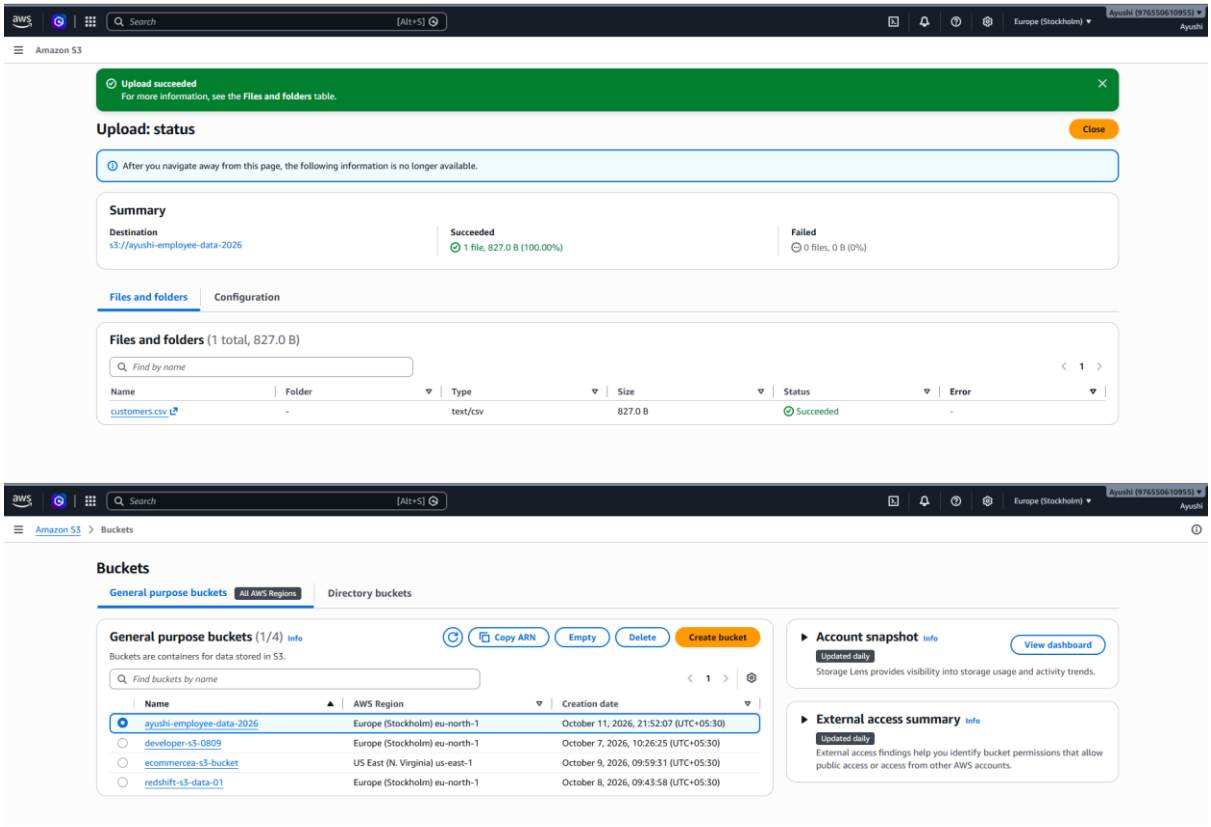
Amazon S3 is used to create a storage bucket for the employee dataset. The bucket provides a location to store the original input data before processing.

- **Uploading the employee dataset**


Search [Alt+S]
Europe (Stockholm)
Ayushi
Amazon S3 > Buckets > ayushi-employee-data-2026 > Create folder
Create folder Info
Use folders to group objects in buckets. When you create a folder, S3 creates an object using the name that you specify followed by a slash (/). This object then appears as folder on the console. Learn more
Your bucket policy might block folder creation
If your bucket policy prevents uploading objects without specific tags, metadata, or access control list (ACL) grantees, you will not be able to create a folder using this configuration. Instead, you can use the upload configuration to upload an empty folder and specify the appropriate settings.
Folder
Folder name
employeea
/
Folder names can't contain "/". See rules for naming
Server-side encryption Info
Server-side encryption protects data at rest.
The following encryption settings apply only to the folder object and not to sub-folder objects.
Server-side encryption
Don't specify an encryption key
The bucket settings for default encryption are used to encrypt the folder object when storing it in Amazon S3.
Specify an encryption key
The specified encryption key is used to encrypt the folder object before storing it in Amazon S3.
If your bucket policy requires objects to be encrypted with a specific encryption key, you must specify the same encryption key when you create a folder. Otherwise, folder creation will fail.
Cancel
Create folder


Search [Alt+S]
Europe (Stockholm)
Ayushi
Amazon S3 > Buckets > ayushi-employee-data-2026
Successfully created folder "employeea".
ayushi-employee-data-2026 Info
Objects
Metadata
Properties
Permissions
Metrics
Management
File systems
Access Points
Objects (1)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Name
Type
Last modified
Size
Storage class
employeea/
Folder

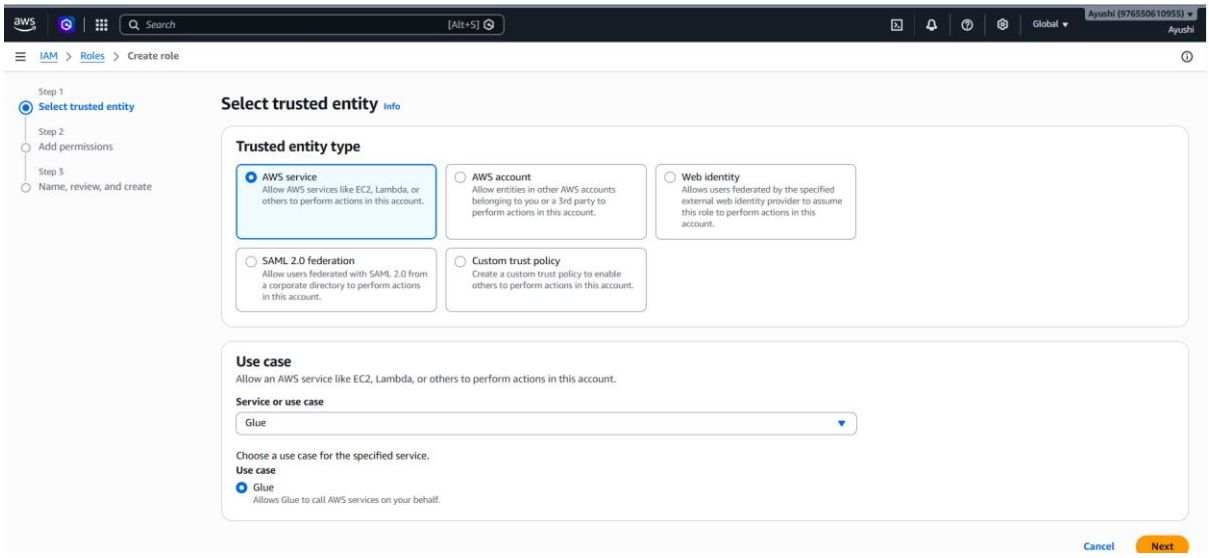

Search [Alt+S]
Europe (Stockholm)
Ayushi
Amazon S3 > Buckets > ayushi-employee-data-2026 > Upload
Upload Info
Add the files and folders you want to upload to S3. To upload a file larger than 160GB, use the AWS CLI, AWS SDKs or Amazon S3 REST API. Learn more
Drag and drop files and folders you want to upload here, or choose Add files or Add folder.
Files and folders (1 total, 827.0 B)
All files and folders in this table will be uploaded.
Remove
Add files
Add folder
Find by name
Name
Folder
Type
Size
customers.csv
-
text/csv
827.0 B
Destination Info
Destination
s3://ayushi-employee-data-2026
Destination details
Bucket settings that impact new objects stored in the specified destination.
Permissions
Grant public access and access to other AWS accounts.
Properties
Specify storage class, encryption settings, tags, and more.
Cancel
Upload



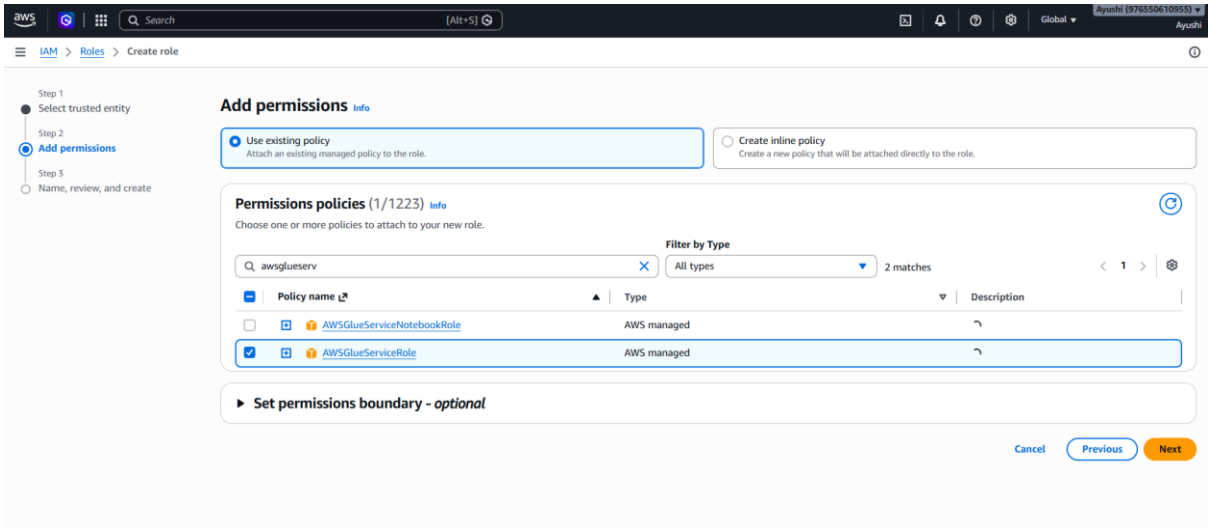
The employee CSV file is uploaded to the S3 bucket's input folder so that AWS Glue can access the raw dataset for further processing.

IAM:

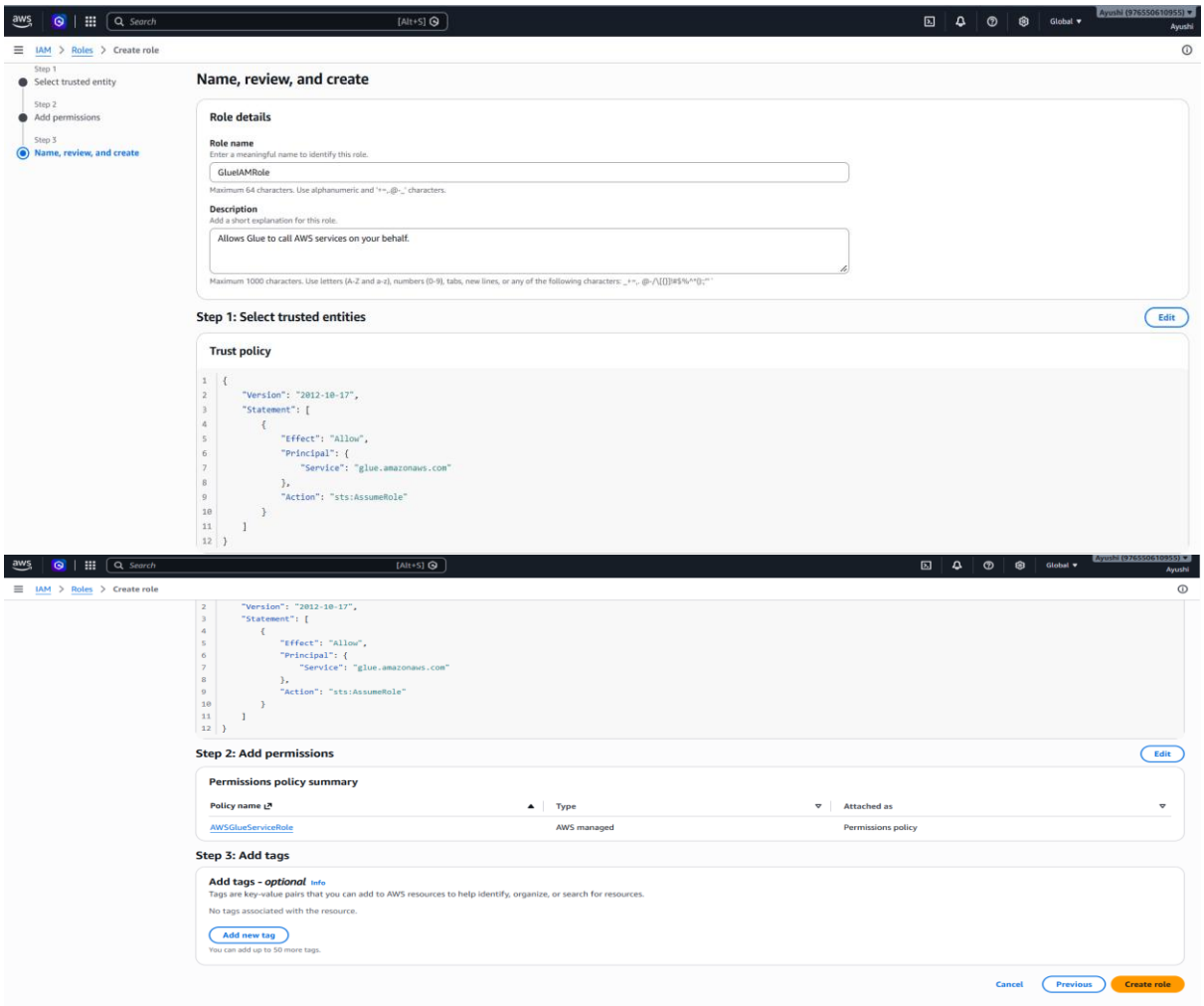
➢ Creating an IAM role



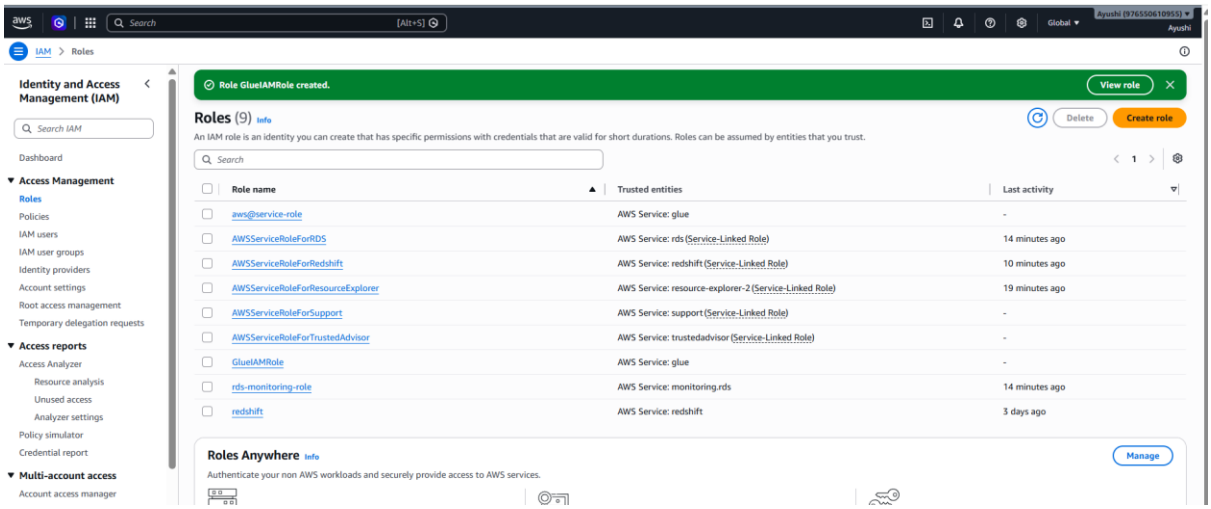
AWS Glue is selected as the trusted service so that it can assume the role and perform authorized operations.



The required permissions are attached to the IAM role to support Glue operations and controlled access to the relevant S3 input and output locations.



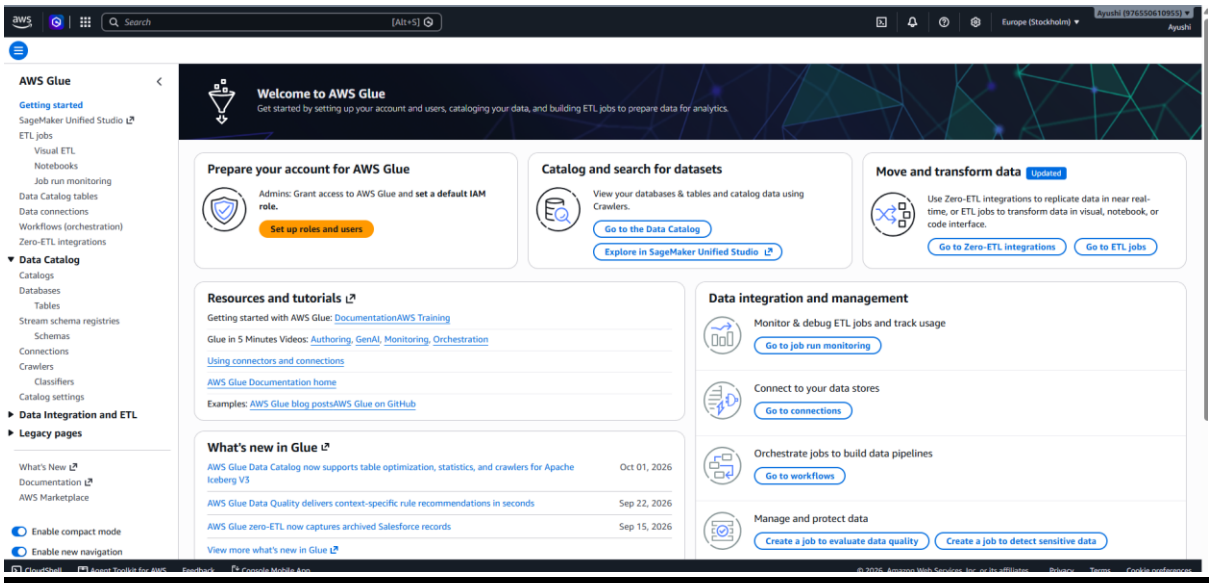
The IAM role configuration is reviewed to verify the trusted service and attached permissions before using the role in AWS Glue.



An IAM role is configured to allow the AWS Glue service to access the AWS resources required for the data processing workflow.

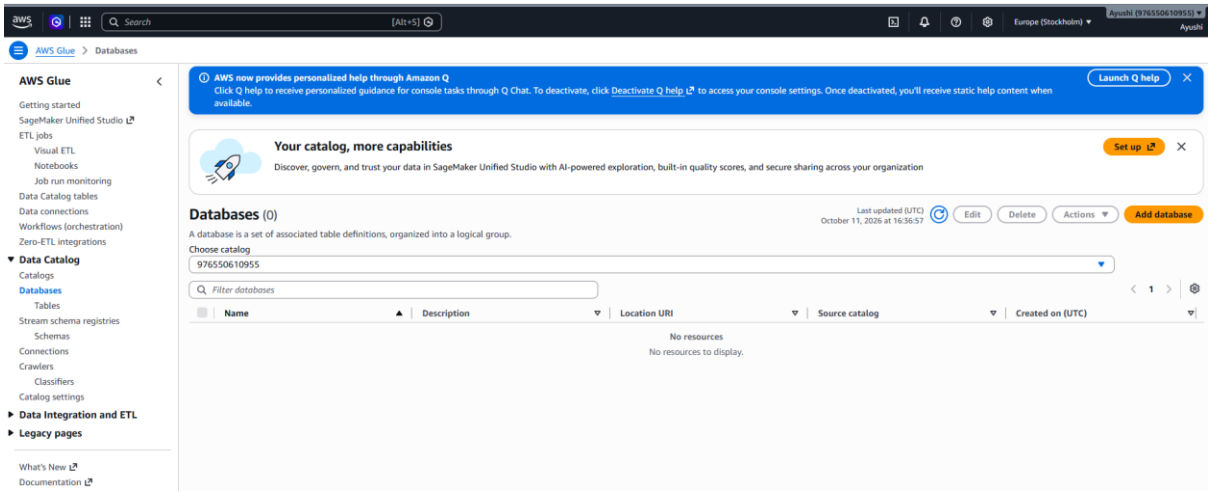
Glue

- **Opening AWS Glue**

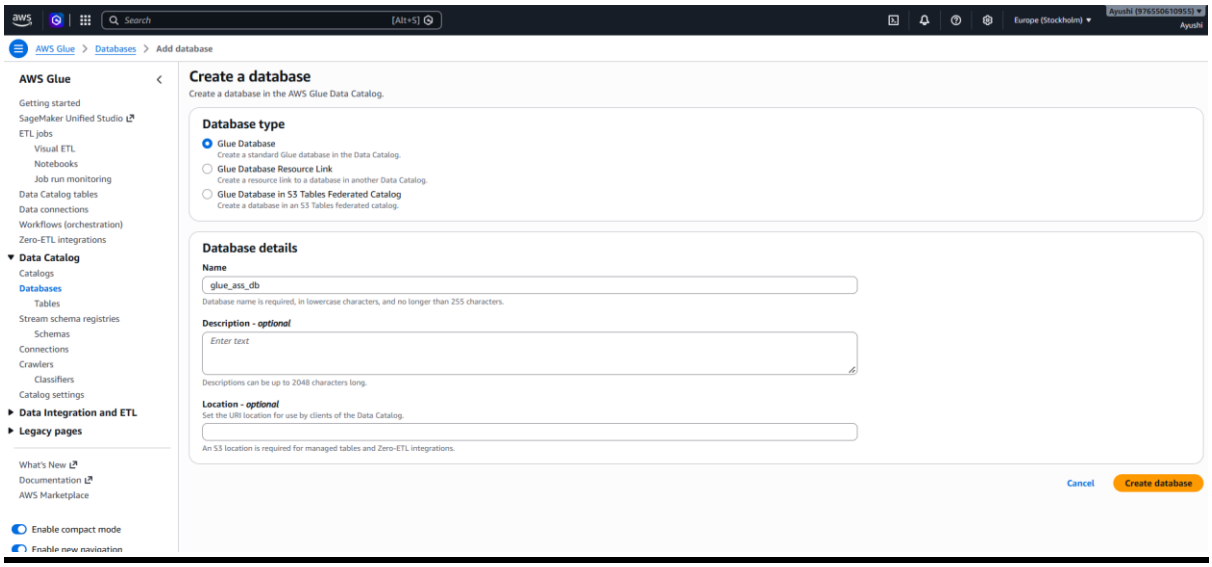


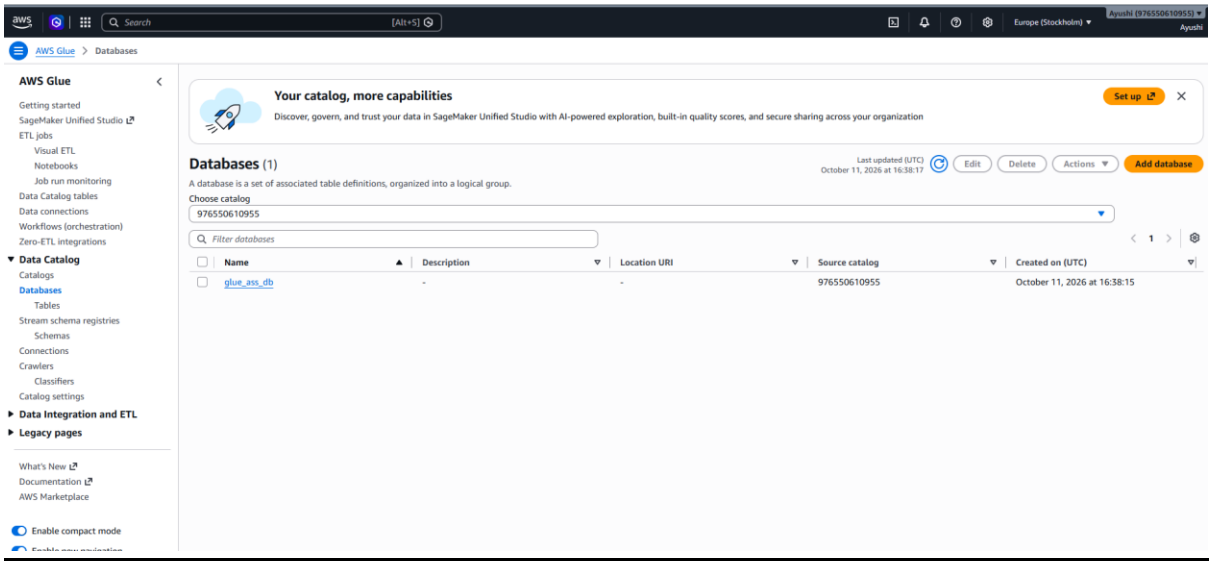
AWS Glue is a managed data integration service used to discover, prepare, transform, and process datasets stored in AWS.

- **Click on Add Database**



- **Select Glue Database and Add Database Details**

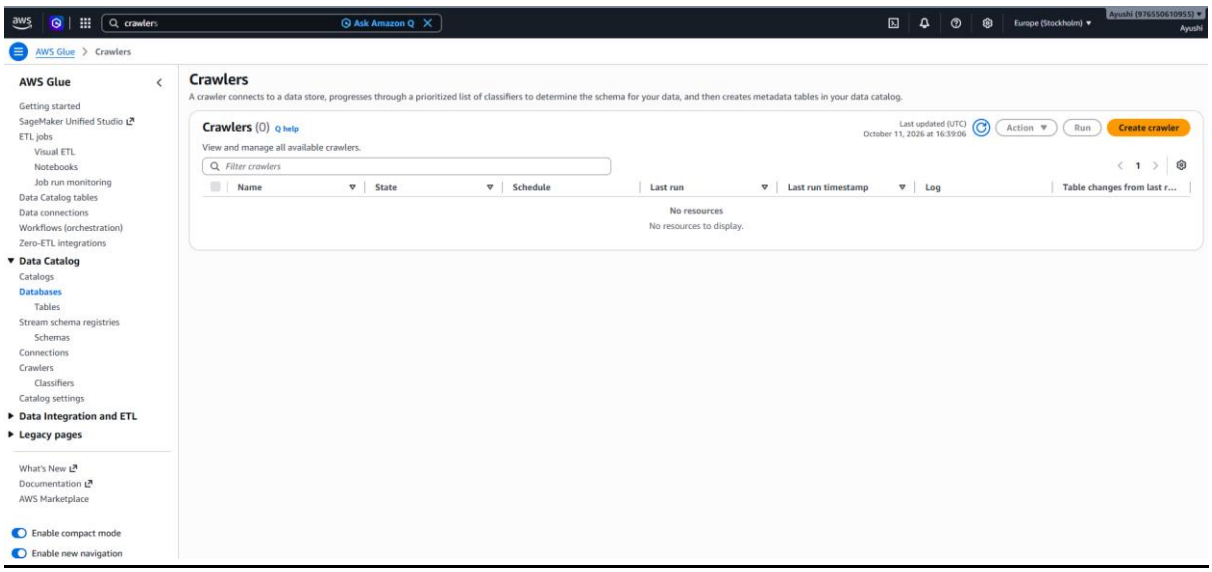




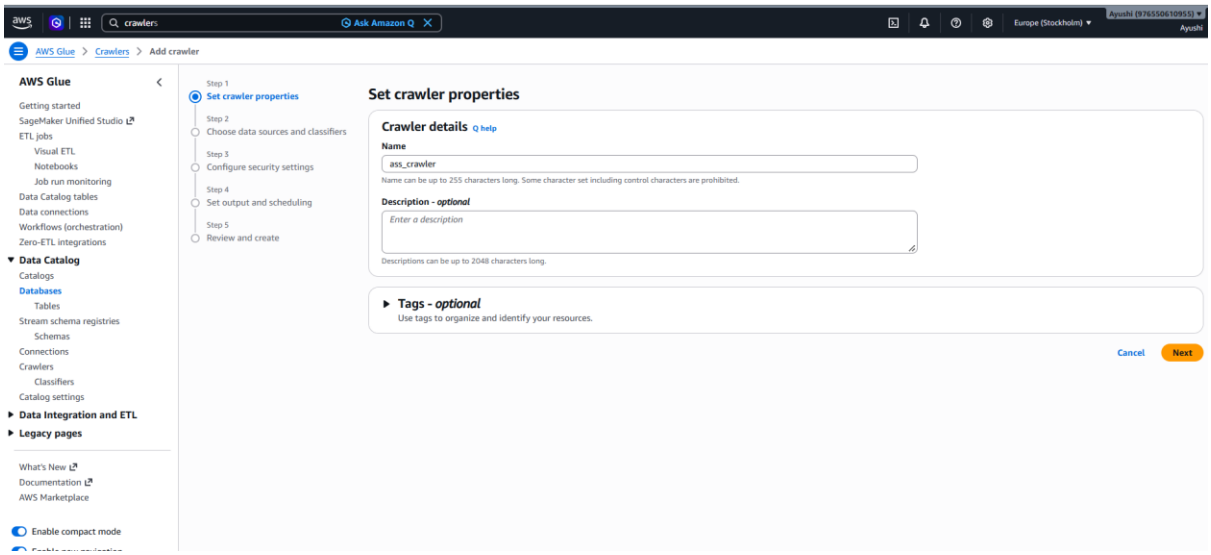
Database is added successfully

AWS Crawler

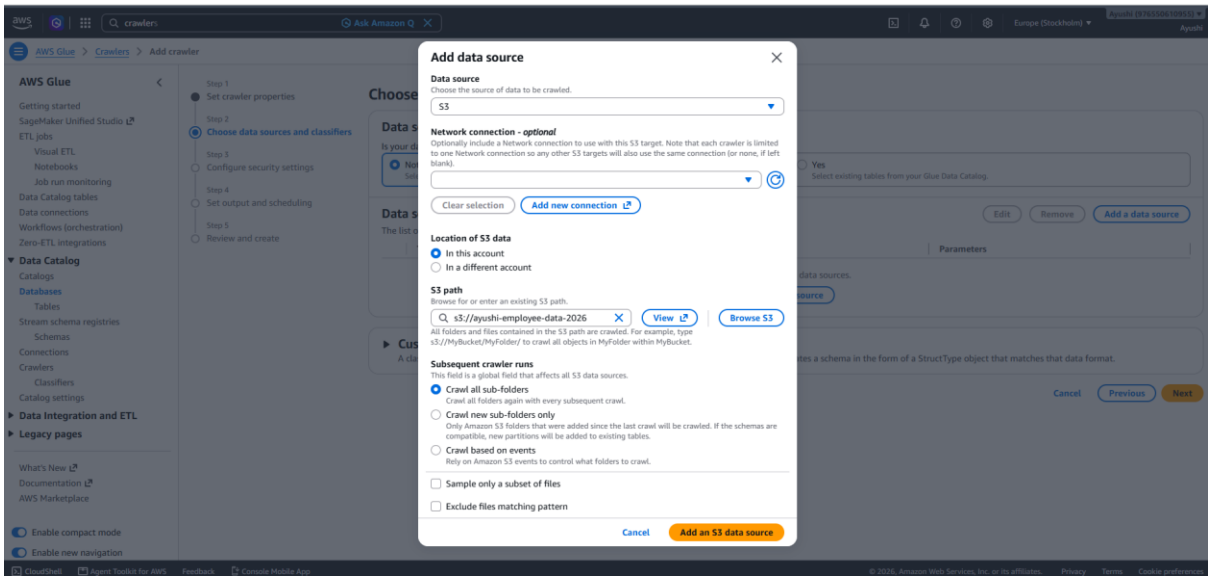
- **Creating the Glue Crawler**



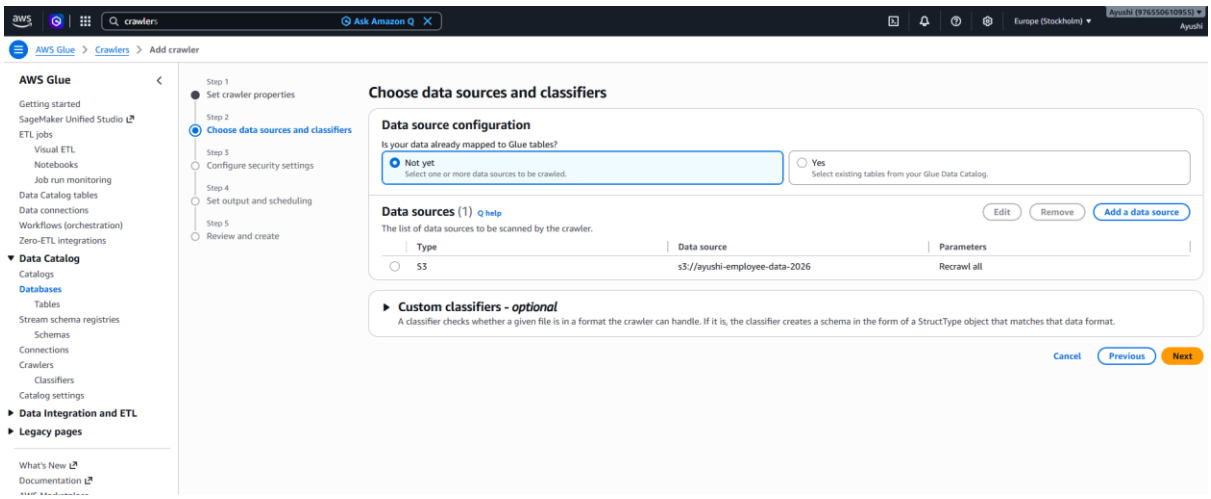
A Glue Crawler is configured to scan the employee dataset stored in Amazon S3 and identify its schema, including column names and data types.



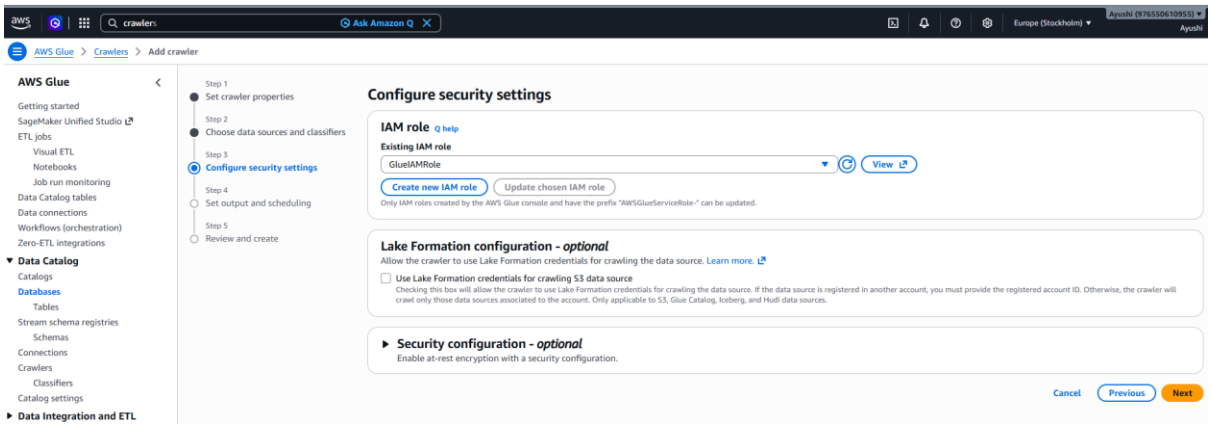
A Glue Crawler is configured to scan the employee dataset stored in Amazon S3 and identify its schema, including column names and data types.



➢ Configuring the crawler data source

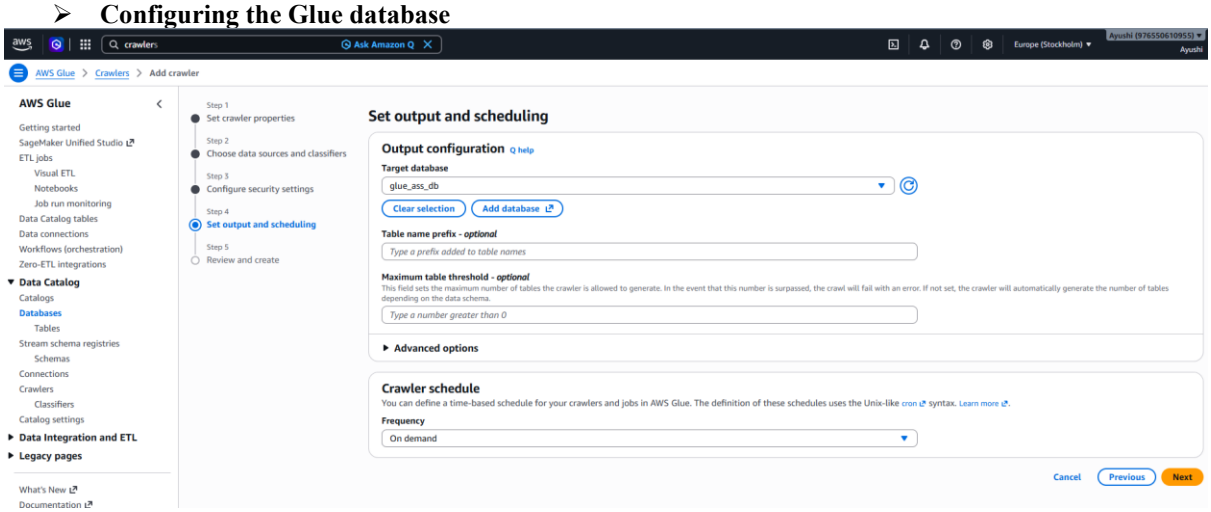


- **Selecting the IAM role for the crawler**



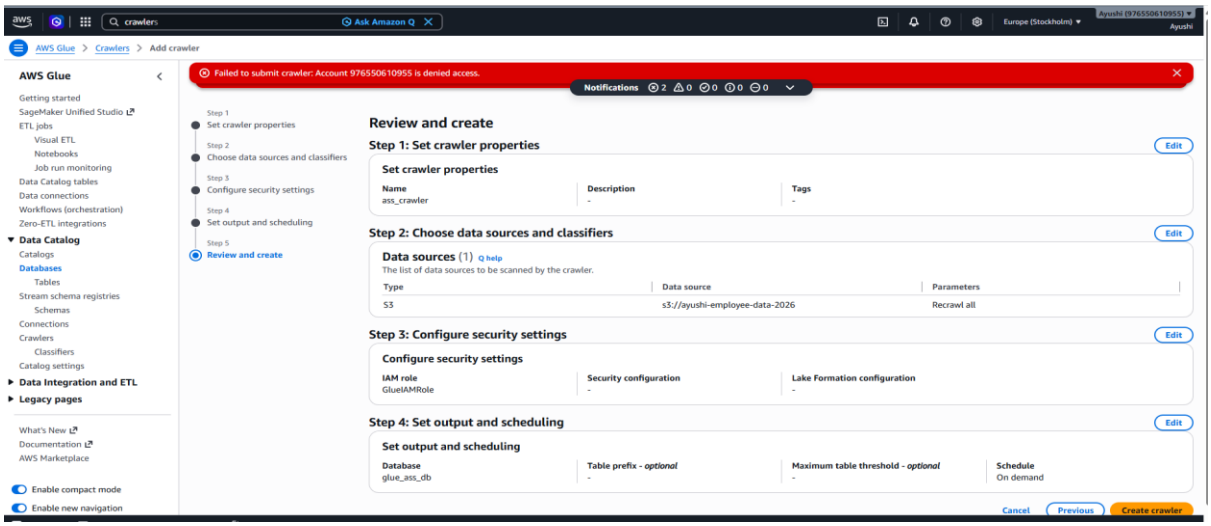
The appropriate IAM role is selected to provide the crawler with permission to access the source data and update the Glue Data Catalog.

- **Configuring the Glue database**



A Glue database is selected or created to organize the metadata tables discovered by the crawler.

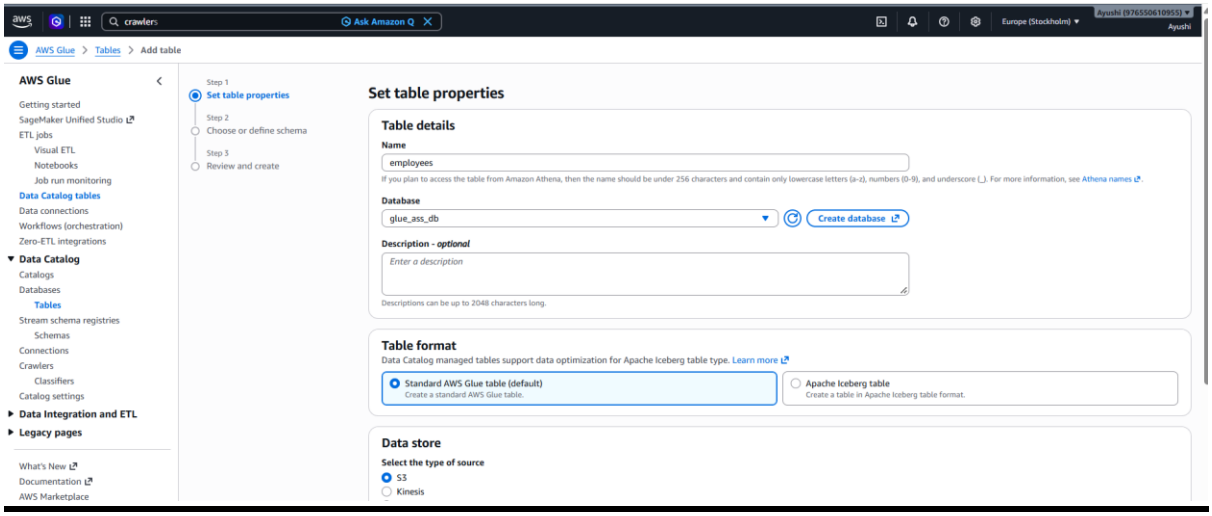
- **Running the Glue Crawler**

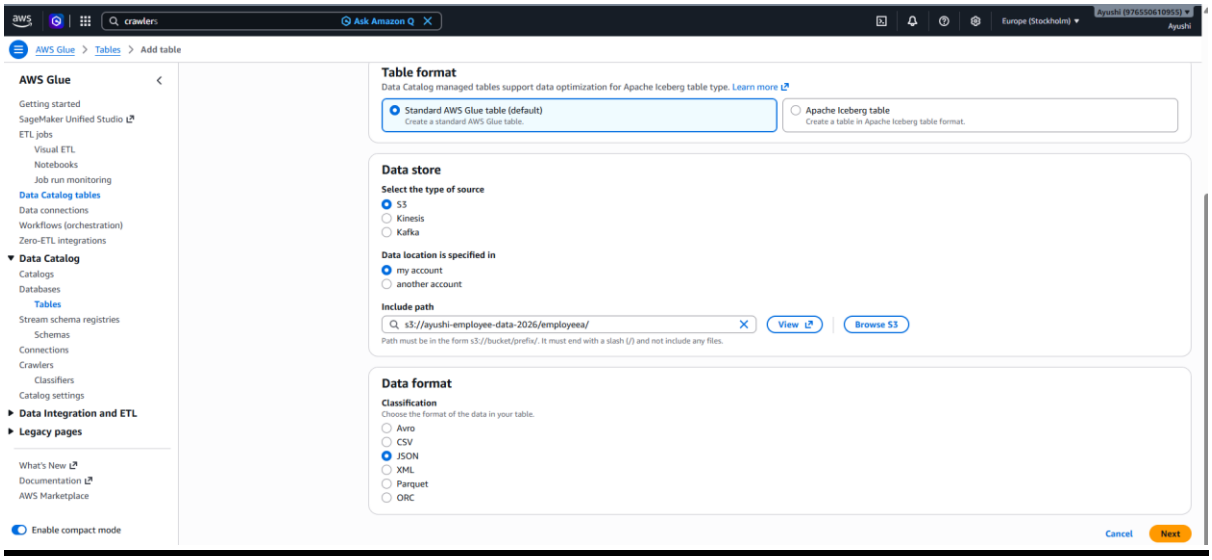


The crawler is run to inspect the source data, infer its schema, and create or update the corresponding metadata table in the Glue Data Catalog.

Glue Data Catalog

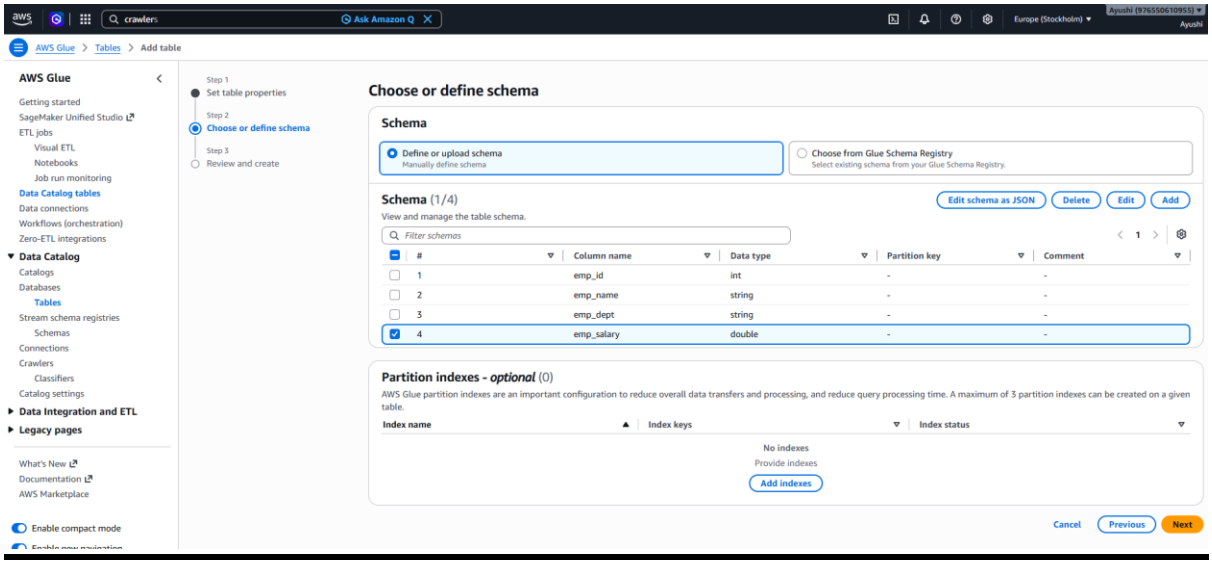
- **Viewing the Glue database and table**



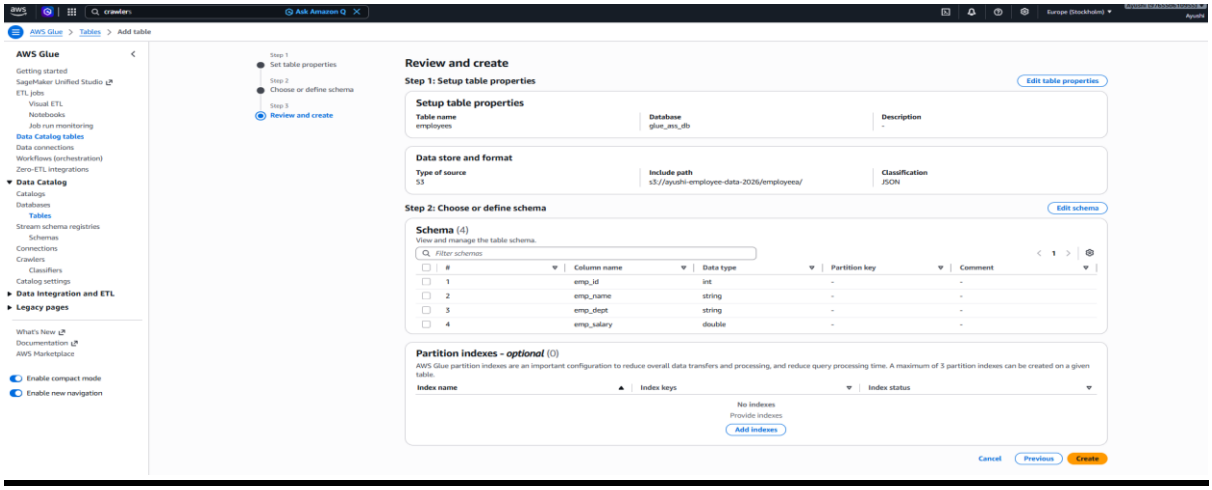


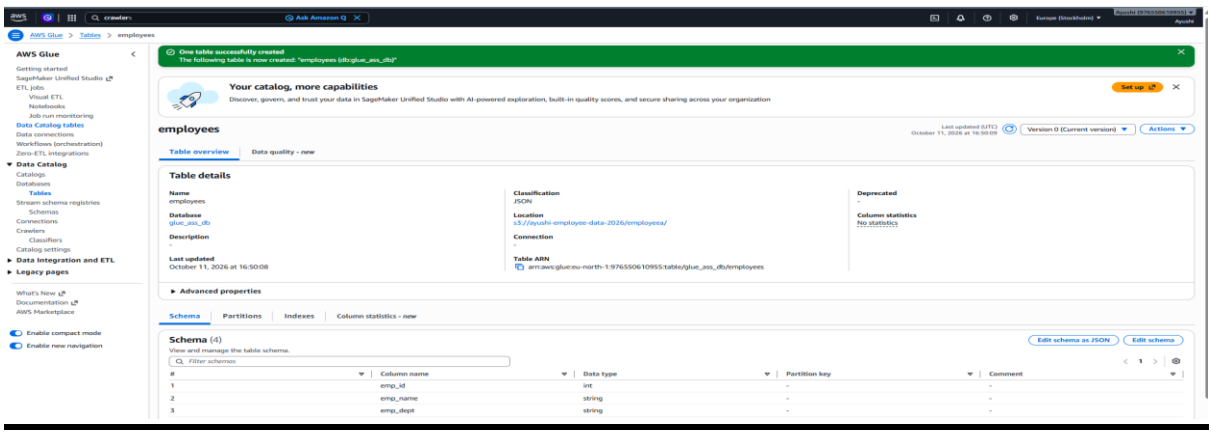
The Glue Data Catalog stores metadata about the employee dataset, including its table definition, column names, and data types. This metadata allows AWS Glue to identify and read the source data.

➢ **Inspecting the table schema**



The table schema is reviewed to verify the discovered columns and their data types before using the dataset in the ETL workflow.

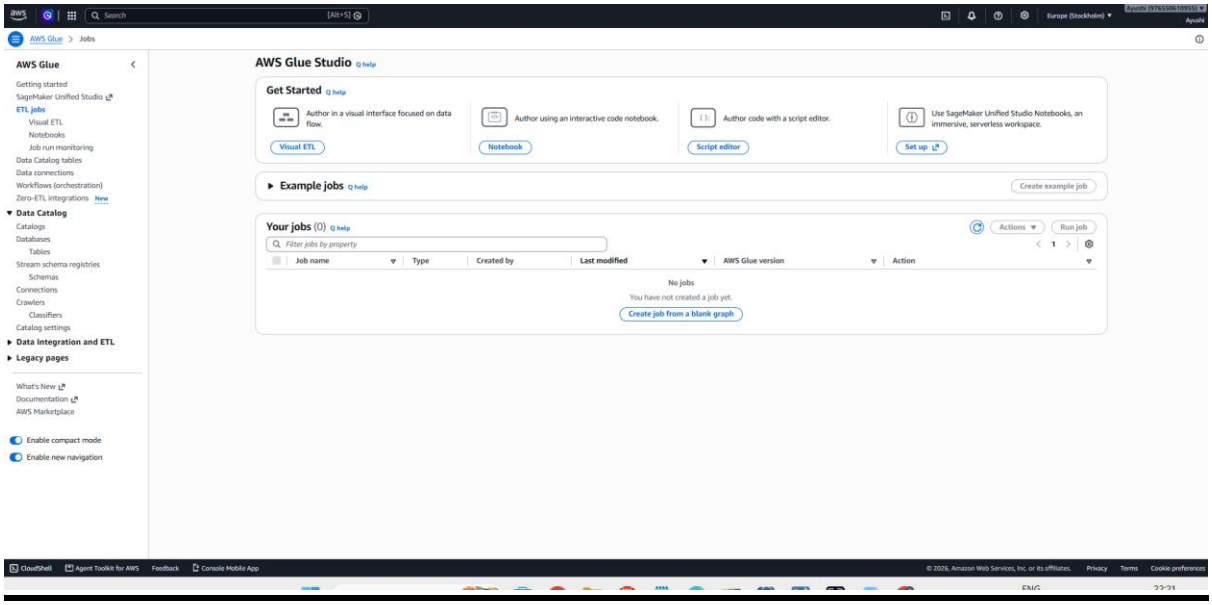




Data Catolog Table is reviewed and created

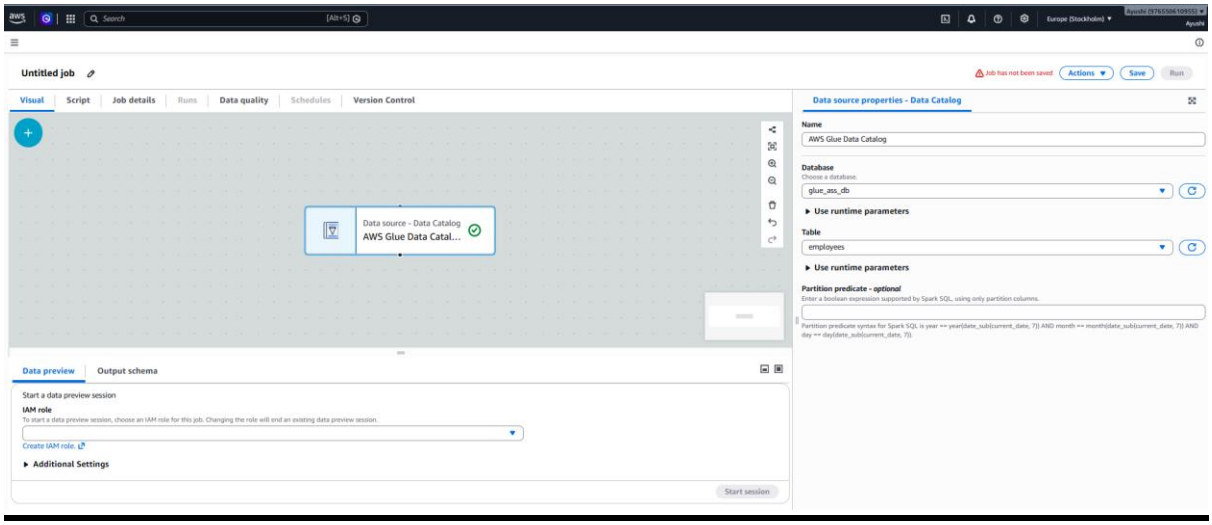
Glue Visual ETL

- **Opening Glue Visual ETL**

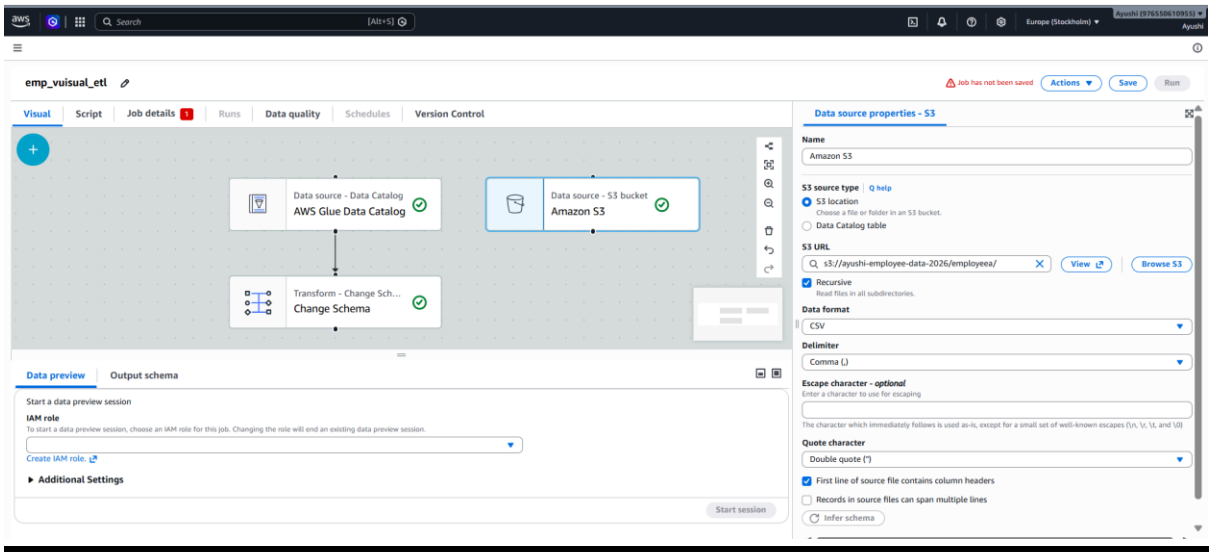


Glue Visual ETL provides a visual interface for building data transformation workflows by connecting source, transformation, and target nodes.

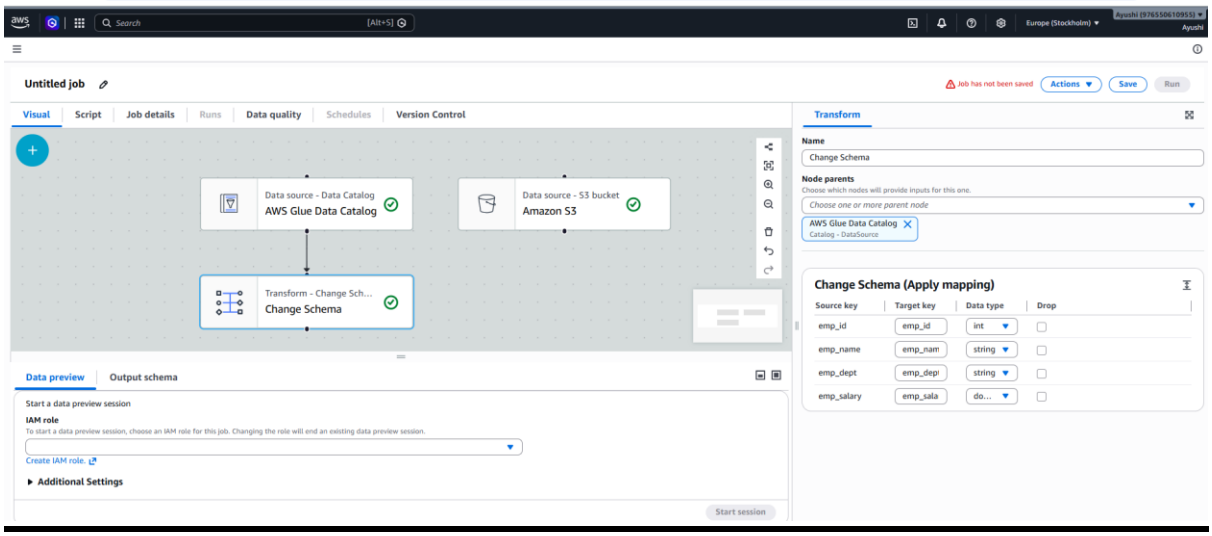
- **Selecting the source dataset**



The employee table from the Glue Data Catalog is selected as the source, allowing the ETL job to read the dataset discovered by the crawler.

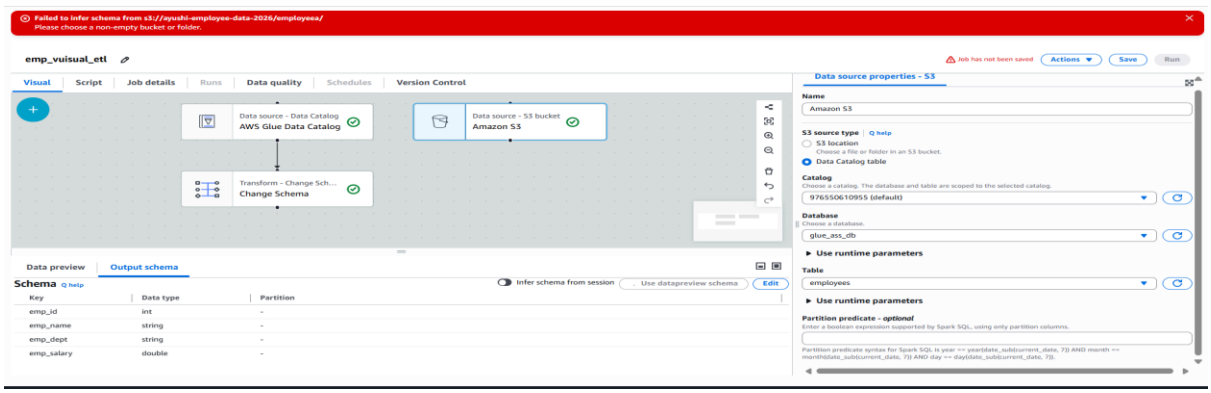


➢ Configuring data transformations



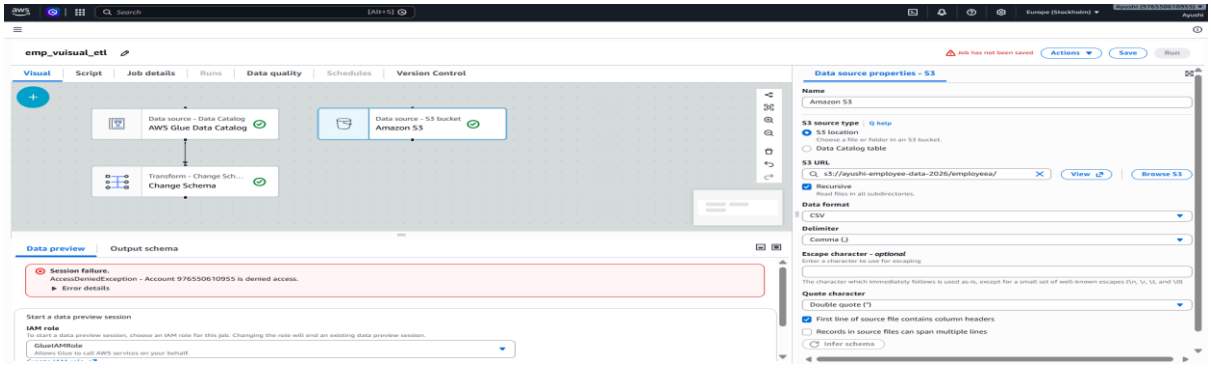
The transformation steps are configured to prepare the employee data according to the requirements of the assignment.

➢ Configuring the target location



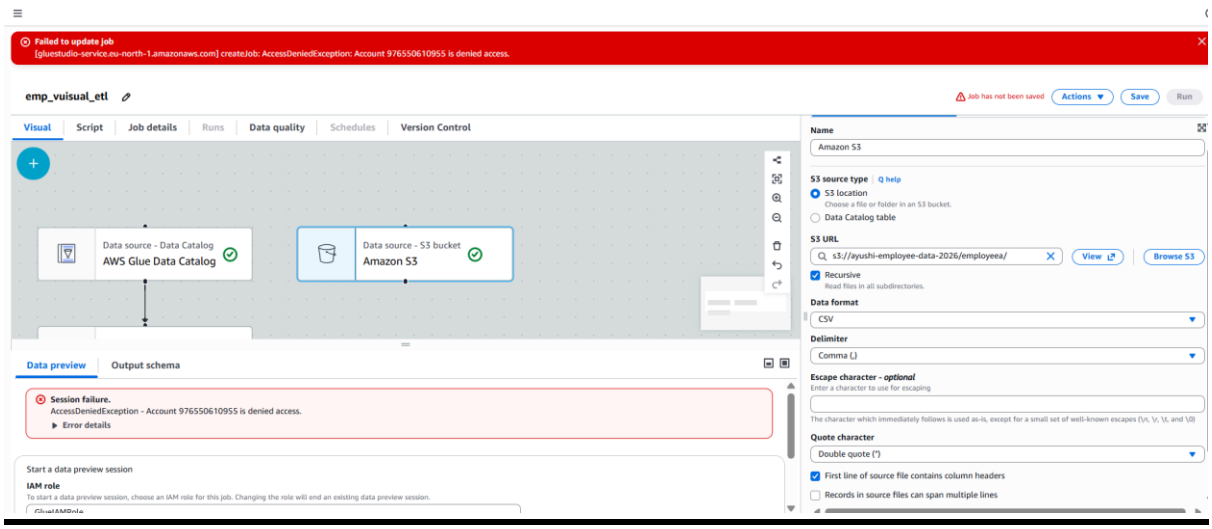
The S3 curated-data location is configured as the target so that the processed dataset can be stored separately from the original input data.

- **Reviewing the ETL workflow**



The visual ETL workflow connects the source dataset to the transformation steps and the target location, representing the complete data processing pipeline.

- **Running or monitoring the ETL job**



The ETL job is executed or monitored to check whether the configured data processing workflow completes successfully.

Final Explanation

This assignment demonstrates the use of SQL, Linux commands, and AWS services for employee data management and cloud-based data processing. SQL queries were used to create a database, insert employee records, retrieve specific information, filter data, and perform aggregate operations.

Linux commands were practiced to manage files and directories, view file contents, and search and organize employee records.

Amazon S3 was used to store the raw employee dataset. AWS IAM provides the roles and permissions required for authorized access to AWS resources. The objective of this workflow is to understand how AWS services work together to store, discover, transform, and organize employee data for further analysis. The complete process starts with raw data in Amazon S3 and ends with processed data stored in a curated S3 location.

1. Amazon S3 – Raw Data Storage
Amazon Simple Storage Service (S3) is used to store the original employee dataset in CSV format. The dataset contains employee-related information such as employee ID, name, department, salary, and city. The raw data is uploaded to the input folder of the S3 bucket, where it remains available for processing. This provides a central storage location for the data used in the assignment.

2. AWS Glue Crawler – Data Discovery
The AWS Glue Crawler scans the dataset stored in the S3 input location. It examines the structure of the file and identifies details such as column names and data types. Based on this information, the crawler automatically discovers the schema of the dataset. This reduces the need to define all the metadata manually and helps prepare the data for further processing.

3. AWS Glue Data Catalog – Metadata Management
The AWS Glue Data Catalog stores the metadata discovered by the crawler. This metadata includes the database name, table name, column definitions, and data types. It provides an organized view of the employee dataset without requiring the raw data to be moved from S3. The catalog table can then be selected as the source when creating the ETL workflow in AWS Glue.

4. AWS Glue Visual ETL – Workflow Creation
AWS Glue Visual ETL provides a graphical interface for designing data integration workflows. The employee dataset registered in the Data Catalog is selected as the source. The required transformation steps and the destination for the output are configured through the visual interface. This makes it easier to understand how data flows from the source through the processing stages to the final destination.

5. Data Transformation – Preparing the Data
During the transformation stage, the configured ETL operations are applied to the employee dataset. Depending on the assignment requirements, these operations may include selecting specific columns, filtering records, changing data types, or organizing the data into a suitable structure. These steps help prepare the data for further use. Only the transformations configured in the ETL workflow are applied to the dataset.

6. ETL Job Execution – Processing the Dataset
After configuring the source, transformations, and destination, the Glue ETL job is run to process the data. AWS Glue reads the source dataset, applies the configured operations, and prepares the transformed output. The job status and execution details can be checked in the AWS Glue console to determine whether processing completed successfully.

7. Amazon S3 – Curated Data Storage
The transformed dataset is written to the designated curated folder in the S3 bucket. This separates the processed data from the original raw input and makes the output easier to manage. The output format depends on the ETL job configuration; for example, Parquet can be used for efficient analytical storage if configured. The curated data can then be used for further analysis and reporting.

8. Final Outcome
This workflow demonstrates how AWS services can be combined to create a data processing pipeline. Amazon S3 provides storage, the Glue Crawler discovers the dataset's structure, and the Data Catalog manages its metadata. Glue Visual ETL defines the workflow, while the transformation steps prepare the data before the

processed output is stored in S3. The result is a structured workflow that supports organized data management and prepares employee information for downstream use.

Complete Workflow:
Amazon S3 (Raw Data) → AWS Glue Crawler → AWS Glue Data Catalog → AWS Glue Visual ETL → Data Transformation → ETL Job Execution → Amazon S3 (Curated Data)